

مت‌سیس درک‌لمع ل‌ماک تادنت‌سم — NovaShop

actual source code + database schema inspected on 2026-08-15. ت‌قی‌ق‌ح ع‌ب‌ن‌م

Statuses used: ☐ ص‌خ‌ش‌م‌ان ☐ ل‌اع‌ف‌ری‌غ ☐ ل‌اع‌ف‌ری‌غ ی‌لو ه‌د‌ش ی‌د‌ن‌ب‌ر‌ک‌ی‌پ ☐ ی‌ئ‌ز‌ج / ص‌ق‌ان ☐ ه‌د‌ش ی‌ز‌اس‌ه‌د‌ای‌پ ☐ ه‌د‌ش‌ن ی‌ز‌اس‌ه‌د‌ای‌پ

No code was modified during this analysis.

ی‌ی‌ار‌ج‌ا ه‌ص‌ال‌خ 1.

NovaShop is a handmade-knitted-doll e-commerce platform built as a 3-tier .NET 9 / Next.js 14 (App Router) application:

- ****Frontend****: Next.js 14 (React, TypeScript, Tailwind, shadcn/ui) on `http://localhost:3000`.
- ****API Gateway****: ASP.NET Core + YARP reverse proxy on `http://localhost:5100` (JWT auth, per-client rate limiting, CORS).
- ****Backend API****: ASP.NET Core minimal-API + MediatR (CQRS) on `http://localhost:5003` (EF Core, SQL Server `(localdb)\mssqllocaldb`, Hangfire, Serilog).
- ****Database****: SQL Server LocalDB `NovaShopDb`. Hangfire storage is in the same database (`Hangfire.\*` schema).

The shop is functional end-to-end for browsing, cart, and order placement, but online payment is disabled by policy (Policy.OnlineEnabled=false), so checkout is InPerson (ی‌ر‌وض‌ح ت‌خ‌ادر‌پ) only. لو‌پ ف‌ی‌ک is also disabled by policy (Enabled=false → 403). SMS uses the real Kavenegar provider (selected via `Sms:Provider=Kavenegar` in User Secrets) but the Kavenegar account currently returns HTTP 412/427 (sender-line / credit issues) — see §15. MassTransit/RabbitMQ is disabled by default (stub `IPublishEndpoint`). OpenTelemetry is wired but exports to an OTLP collector that is not present in this environment.

مت‌سیس ی‌رام‌ع‌م 2.

```
Browser / Next.js (localhost:3000)
| fetch via API_GATEWAY_URL
▼
API Gateway (YARP, :5100)
- JWT bearer validation (Jwt:Key from User Secrets)
- Global rate limit (600/min) + per-route auth rules
- CORS (allowed origin http://localhost:3000)
- Correlation-Id middleware
- Reverse-proxy → backend-cluster (http://localhost:5003)
▼
Backend API (MediatR CQRS, :5003)
- Minimal API endpoint modules
- Pipeline: ExceptionHandler → RateLimiting → CORS → Auth → Antiforgery → Hangfire
- MediatR: ValidationBehavior (FluentValidation) → LoggingBehavior → Handler
- Application Services: NotificationService, KavenegarSmsService, لو‌پ ف‌ی‌ک Service
- Domain (EF Core entities + domain methods) → NovaShopDbContext
▼
SQL Server (localdb) NovaShopDb + Hangfire storage (same DB)
```

Cross-cutting components:

- ****Hangfire****: SQL Server-backed; runs 6 recurring jobs (see §16) on queues `critical/default/notifications/sms/maintenance`.
- ****MassTransit/RabbitMQ****: ☐ DISABLED by default (`Development:EnableMassTransit` defaults to `false`).
`OrderCreatedEvent` / `StockReservedEvent` are published to a ****stub**** `IPublishEndpoint` (no-op). Real RabbitMQ consumers (`ProductCreatedConsumer`, `OrderCreatedConsumer`, `StockReservedConsumer`) exist but are not registered.

- **SMS**: `NotificationService` → `ISmsService` (Kavenegar / Log / Mock selected by `Sms:Provider`).
- **تخادير Mock Gateway**: always (no real PSP). Online payments disabled by policy.
- **External services**: Kavenegar (SMS REST API). No other external calls in the running configuration.
- **Caching**: In-memory `IMemoryCache` (Redis only if `Cache:Provider=Redis` configured — not active).
- **Logging**: Serilog → Console + daily rolling file (`logs/novashop-.log`). OpenTelemetry → OTLP (collector not present).

ناربراک یاهشون 3.

Two roles exist in the `User` entity: `Admin` and `Customer`.

Role	Auth	Pages	API scope
Admin	Must login with password	login, register, dashboard, orders, notifications, product, category, review, etc.	Full control
Customer	Can login with phone number or password	login, register, product, category, review, etc.	Read-only access to public endpoints

Authorization is enforced by a single policy `AdminOnly` = `RequireRole("Admin")`. Customers are blocked from admin endpoints (403). JWT carries `ClaimTypes.Role` = `user.Role`.

Login admin vs customer: the ONLY difference is the `Role` claim. There is no separate admin auth endpoint — `/admin/login` calls the same `/api/auth/login` and stores the token in `localStorage` + cookie. The `/admin` panel does not appear in the nav for non-admins (the customer `layout` guards admin links by role).

تیوه زارح 4.

Endpoints (`AuthEndpoints.cs`, all under `/api/auth`, AllowAnonymous except `/logout`):

- `POST /api/auth/login` — username + password → `LoginResponse{Token, RefreshToken, Expires}` (8h JWT).
- `POST /api/auth/register` — username, phoneNumber (`^09\d{9}\$`), password → stages pending registration + sends OTP SMS. Returns `{pending:true}`.
 - `POST /api/auth/register/resend` — resend OTP (60s cooldown enforced client-side).
- `POST /api/auth/register/verify` — phoneNumber + 6-digit code → verifies OTP, creates `User` (Role=Customer, Email=`{phone}@novashop.local`), returns JWT.
 - `POST /api/auth/otp/request` — existing user only (throws 404 if not found); sends login OTP SMS.
 - `POST /api/auth/otp/verify` — phoneNumber + code → returns JWT for existing user.
- `POST /api/auth/refresh` — refresh token (Accept: AllowAnonymous; handler issues new token).
- `POST /api/auth/logout` — requires auth; returns "Logged out" (client-side cookie clear; server is stateless — no token revocation/blacklist).
- `POST /api/auth/check-mobile` — existence-only (`{exists:bool}`), identical shape for found/not-found (anti-enumeration).

Flows:

1. **Password login**: client posts `{username,password}` → `LoginCommandHandler` verifies PBKDF2 hash → issues HS256 JWT (8h, `Issuer/Issuer=NovaShop`). Token stored in `token` cookie (8h) by `AuthProvider.signIn`.
2. **OTP login** (existing users): `/otp/request` (SMS) → `/otp/verify` → JWT. `RequestOtpCommandHandler` only sends to existing active users.
3. **Registration**: `/register` → `RegisterCommandHandler` stores pending registration in `PendingRegistrationStore` (in-memory singleton) + OTP in `OtpStore` (in-memory singleton) → sends OTP SMS → `/register/verify`

(`VerifyRegistrationCommandHandler`) creates the user and issues JWT.

OTP storage: `OtpStore` and `PendingRegistrationStore` are in-memory singletons (process-scoped). They survive within one API process lifetime but are lost on restart, and are NOT shared across multiple API instances. There is no OTP expiry timestamp checked in the code path shown (a `CanRequest` rate guard exists, but the verify path uses `TryVerify` without an explicit TTL check visible). This is a limitation for multi-instance deployments.

Refresh token: `LoginResponse.RefreshToken` is a random GUID; `RefreshTokenHandler` exists but the store is in-memory / not persisted — refresh is effectively best-effort (no DB refresh-token table found). The FE primarily relies on the 8h JWT cookie.

Logout: client deletes the cookie/localStorage; the server endpoint does nothing server-side (no revocation).

Redirect after login: FE `router.push('/products')` for both customer and admin-login (then admin layout routes to `/admin`).

5. یرتشم تاناکم

□ هدش یراسهدای پ (verified in FE + BE):

- Register (phone+OTP), login (password or OTP), logout.
- Browse products (home + `/products` with search/filter/sort), product detail page (`getProductById`, includes category + reviews).
- Product search/suggestions (full-text via Dapper `SearchProductsQuery`, `GetProductSuggestionsQuery`, FTS catalog rebuilt daily by Hangfire).
 - دیرخ دبس (add/remove/update-qty/clear) — DB-backed per user.
 - باسح هیوس (address + shipping method + discount + InPerson payment) → create order.
 - My orders list + order detail.
 - اهنالعا (in-app list, mark read, mark all read, unread count).
 - Wishlist (add/remove — `WishlistEndpoints` + handlers).
 - Reviews (create/delete — `ReviewsEndpoints`; product detail shows reviews).
 - Custom doll request (create + my requests + accept approved price).
 - لوپ فیک page exists but is **□ disabled** (403) — see §21.
- Profile page (`/profile`) — reads user from JWT; no backend profile-update endpoint found (the FE shows info; editing is □ not wired to a verified backend command).

△ یئزج / صقان / notes:

- "Select variant" maps to **ProductColor** (per-product color with its own stock). Add-to-cart requires `productColorId` when the product has colors (validated in `AddToDبس دیرخCommandHandler`).
- Tracking order: the order detail page shows status + (if admin set) tracking code; customer-facing live shipment tracking is the order status only.

6. تالوصحم تیری دم

Backend (`ProductsEndpoints`, `CategoriesEndpoints`, `ReviewsEndpoints`, `BannersEndpoints`):

- `GET /api/products` (public), `GET /api/products/{id}` (public), `GET /api/products/search`, `GET /api/products/suggestions` (public).

- `POST/PUT/DELETE /api/products` — ****AdminOnly****.

- Categories: `GET/POST/PUT/DELETE /api/categories` (AdminOnly for writes).

- Reviews: `POST/DELETE /api/reviews` (customer create; admin delete).

- Banners: `GET /api/banners` (public), admin CRUD under `/api/admin/banners`.

CreateProductCommand supports: Name, Description, Price, OriginalPrice, ImageUrl, Stock, CategoryId, Images (list of `ProductImageInput{Url,AltText,DisplayOrder,IsPrimary,ProductColorId}`), Colors (list of `ProductColorInput{Name,HexCode,Stock,IsActive,Price}`). FluentValidation enforces Name≥3, Price>0, Stock≥0, ImageUrl required, CategoryId>0.

Product entity (`Product.cs`): Id, Name, Description, Price, OriginalPrice, ImageUrl, Rating, Stock, Category(FK), Images (`ProductImage`), Colors (`ProductColor`), Reviews. Domain methods: `ReserveStock`, `ConfirmReservation`, `ReleaseReservation` (stock bookkeeping for checkout).

ProductColor (defined in `ProductImage.cs`): Id, Name, HexCode, Stock, IsActive, Price, Images — color-specific stock + images.

ProductImage (`ProductImage.cs`): Id, Url, AltText, DisplayOrder, IsPrimary, ProductColorId?. `Product.PrimaryImageUrl` resolves primary/first image.

Functionality	Frontend	Backend	DB	Admin	Status
List products	☐	☐	☐	—	☐
Product detail	☐	☐	☐	—	☐
Search/filter/sort	☐	☐	☐	—	☐
Suggestions (autocomplete)	☐	☐	☐	—	☐
Create product	admin page	☐	☐	☐	☐
Edit product	admin page	☐	☐	☐	☐
Delete product	admin page	☐	☐	☐	☐
Multiple images	admin page	☐	☐	☐	☐
Color variants + color stock	☐ (select)	☐	☐	☐	☐
Color-specific images	FE type	☐	☐	☐	☐
ی‌دوچوم (stock)	shown	☐	☐	☐	☐
Categories CRUD	admin page	☐	☐	☐	☐
Reviews	☐	☐	☐	☐ (delete)	☐
Banners CRUD	admin page	☐	☐	☐	☐
Product status (active/inactive)	☐	Stock>0 ⇒ IsAvailable	☐ (availability derived from stock, no explicit status field)		

دیخ دبس 7.

Endpoints (`دیخ دبس` Endpoints, RequireAuthorization): `GET /api/cart`, `POST /api/cart` (AddTo دبس دیخ Request{ProductId,Quantity,ProductColorId?}), `PUT /api/cart/items/{id}`, `DELETE /api/cart/items/{id}`, `DELETE /api/cart` (clear).

Flow:

- دیخ دبس is ****DB-backed**** (per `UserId` via `دیخ دبس` table + `دیخ دبس` Items). The FE `cart-context` mirrors it client-side but the source of truth is the server (`GET /api/cart` on load).
- Add-to-cart: `AddTo دبس دیخ CommandHandler` loads/creates the user's cart, adds item; ****rejects if product has colors but no `productColorId`****, validates stock via `Product.IsAvailable`/color stock.
- Update qty / remove / clear operate on the user's cart.
- ****ی‌دوچوم validation**** happens at add-to-cart (availability) and again at order creation (reservation).

- **transition**: `CreateOrderFrom` دېس د `Command` reads the cart, reserves stock, creates the order, then **deletes the cart** (`_context.Remove(cart)`). FE also clears its local cart cache.

باسج هیوست 8.

Frontend (`/checkout`): form with customer info (name, email, phone, address, city, postalCode), shipping method selector (`POST /COURIER PICKUP`), optional discount code, submit → `POST /api/orders`.

cost (client-computed, sent to backend):

- `PICKUP` → 0
- `COURIER` → 129,000 Toman
- `POST` → 0 if subtotal ≥ 500,000, else 59,900 Toman

The backend accepts `shippingCost` as provided (no server-side recomputation/validation of the amount beyond ≥0).
method is hardcoded `InPerson` in the FE request body.

method enforced by `CreateOrderFrom` دېس د `CommandValidator`: when `Policy.OnlineEnabled=false`, only `"InPerson"` is accepted (else 400 "تخادرپ نیالنه اتقوم" "تس").

Status: `InPerson-only` path). Online payment path exists in code but is disabled by policy.

اهشرافس 9.

Endpoints (`اهشرافس`): `POST /api/orders` (create from cart), `POST /api/orders/{id}/pay`, `GET /api/orders` (mine), `GET /api/orders/{id}` (mine or admin), `POST /api/orders/{id}/cancel` (customer), `POST /api/orders/{id}/refund` (admin), `POST /api/orders/{id}/return-request` (customer).

Creation (`CreateOrderFrom` دېس د `CommandHandler`):

1. Idempotency check (by `Idempotency-Key` header) — returns existing order if present.
2. Loads cart; throws if empty.
3. Persists contact `PhoneNumber` on user (if missing).
4. Begins DB transaction; **reserves stock** (product + color) for 15 min; computes total = cart total + shipping + discount.
5. Creates `Order` (Status=Pending, `Status=Pending`, `Method` forced `InPerson`), `OrderItem`'s, `تخادرپ` (Pending), initial `OrderStatusHistory`, `Transaction` (Reserve) rows.
6. Commits; assigns tracking code `NS-yyyy-#####`; schedules Hangfire expiry; publishes `OrderCreatedEvent` + `StockReservedEvent` (to stub); sends order-placed SMS/in-app notification.
7. Returns `OrderDto`.

Order number / tracking: `TrackingCode = NS-{yyyy}-{Id:D6}` (auto). No separate human order number beyond the DB Id.

Order statuses (from `Order.cs` constants + `ValidTransitions`):

`Pending` → `Confirmed` → `Processing` → `Paid` → `ReadyForPickup` → `Shipped` → `Delivered`, plus `Cancelled`, `Refunded`, `Failed`, `ReturnRequested` → `ReturnApproved` → `Returned`. Transitions are strictly validated (`IsValidTransition`); illegal transitions throw.

deduction timing: دوجوم

- At creation: stock is **reserved** (Stock decreased, ReservedQuantity increased), not yet permanently deducted.
- On cancellation (pre-paid): reservation **released** back to Stock.
- On cancellation (paid/shipped/delivered): **Stock restored** (permanent deduction had occurred at Paid).
- `ReleaseExpiredReservationsJob`` (Hangfire, every 5 min) releases reservations past `ReservationExpiresAt`` back to Stock.
- Note: the handler does not call `ConfirmReservation()` on Paid; the code comments indicate reserved stock becomes permanent at Paid, but `MarkAsPaid`` does not zero `ReservedQuantity`` — this is a Δ partial gap (stock math relies on reservation bookkeeping that is not fully closed on Paid; inventory ledger still records correctly).

Cancellation: customer `POST /api/orders/{id}/cancel`` → `UpdateOrderStatusCommand(Status=Cancelled)`` ; releases/restores stock per rules above; SMS sent.

Return: customer `POST .../return-request`` → `ReturnRequested`` ; admin can `ReturnApproved`/Returned`` ; `Returned`` → `Refunded`` .

Refund: admin `POST .../refund`` → `RefundOrderCommand`` → marks `Refunded`` , refunds to wallet ( `فیک`` `Service`` ) if policy allows. `فیک`` disabled ⇒ refund path is Δ partially functional (would credit a disabled wallet).

Delivery: admin sets status `Shipped`` (assigns random 12-digit `TrackingNumber`` + tracking code) and `Delivered`` via `PUT /api/admin/orders/{id}/status`` .

10. لاسرا

methods (`Order.لاسراPost/COURIER/PICKUP``):

- **POST** (زاتش ی تسپ) — free over 500k, else 59,900.
- **COURIER** (یروتوم کیپ) — 129,000.
- **PICKUP** (یروض لیوحت) — free; no address required (validation skips address for PICKUP).

Selected by customer in checkout (`shippingMethod`` ). Stored on `Order.لاسراMethod`` + `لاسراCost`` . For PICKUP, `PickupLocation`/PickupInstructions`` stored. لاسرا cost is client-computed and passed through — no server-side shipping-rate engine; the admin cannot reconfigure rates from the UI (rates are hardcoded in the FE `checkout/page.tsx``).

Status: ☐ `لاسرا` (POST/COURIER/PICKUP selectable; cost handling as above). Δ لاسرا cost logic lives in FE, not backend — a limitation.

11. تخادرپ

Current method: InPerson (یروض تخادرپ) only. Online payment is ☐ DISABLED (`Policy.OnlineتخادرپEnabled=false``).

Evidence:

- `CreateOrderFromدبسدیخCommandHandler``: `isInPerson = !OnlineتخادرپEnabled || Method=="InPerson"`` → forces `InPerson`` .
- `CreateOrderFromدبسدیخCommandValidator``: when disabled, only `InPerson`` allowed.
- `appsettings.json`` `Policy`: { `OnlineتخادرپEnabled: false`, `فیک`Enabled: false`, `InPersonتخادرپEnabled: true`, `OrderCreationEnabled: true` } `.

- `ConfigureGateway` registers `MockGateway` (always). No real PSP (ZarinPal/IDPay) integration exists in running config.
- `/api/orders/{id}/pay` exists (`ProcessCommandHandler`) and `VerifyCommand` (PSP callback) exists, but with online payments disabled they are not reachable through normal checkout. The `MockGateway` + `/api/mock-gateway/{authority}/complete` endpoints simulate a PSP for dev only.
- status (`Order.Pending/Paid/Failed/Refunded/Expired`): order is created `Pending`; becomes `Paid` only via `MarkAsPaid` (currently only through the disabled online-payment path or admin). For InPerson, payment is effectively settled out-of-band (COD/cash) — the system does not record InPerson payment capture.
- Records: `Transaction` entity per order (Amount, Method, Status, TransactionId). `Transaction` ledger for wallet (disabled).
- Refunds: `RefundOrderCommand` → status `Refunded`, refund to wallet. ⚠️ functional (wallet disabled).
- Status: ☐ Online payment DISABLED · ☐ InPerson path · ☐ Mock gateway (dev) · ☐ Real PSP NOT IMPLEMENTED.

12. دوجوم

- `Product.Stock` (int) + `ProductColor.Stock`; `ReservedQuantity` / `ReservedUntil` on Product.
- **Reservation** at order creation (15-min window) via `ReserveStock`.
- **Release** on cancel (pre-paid) or on expiry (Hangfire `ReleaseExpiredReservationsJob`, every 5 min) via `ReleaseReservation`.
- **Restock** on paid-order cancel via `UpdateOrderStatusCommandHandler` (adds back `item.Quantity`).
- **Ledger**: `Transaction` (Type=Reserve/Release, Quantity, StockBefore/After, Reference) — admin-viewable at `GET /api/admin/inventory`.
- `HealthCheckJob` (Hangfire, every 30 min) — runs a health check (logic not deeply verified; logs/acts on anomalies).
- Low-stock threshold exists in config (`Jobs:LowStockThreshold=5`) but no automatic low-stock notification/alert was verified beyond the health-check job.

Status: ☐ reserve/release/ledger. ⚠️ `ConfirmReservation` not called on Paid (reserved qty not zeroed) — see §9 note.

13. اهنالغا

Two channels:

1. **In-app** (`AppNotification`): created by `NotificationService.NotifyInAppAsync` for order-placed, payment-successful, status-changes, custom-doll events. Stored with `UserId, OrderId?, CustomDollRequestId?, Type, Channel=InApp, Title, Message, Status=Sent, IsRead, ReadAt`.
2. **SMS** (`SmsNotification`): see §15.

Endpoints (`Endpoints`, RequireAuthorization): `GET /api/notifications` (mine, paged), `POST /api/notifications/{id}/read`, `POST /api/notifications/read-all`, `GET /api/notifications/unread-count` (header bell).

Admin SMS log: `GET /api/admin/notifications/sms` (paged, filter by orderId/status).

Order status changes trigger `NotifyOrderStatusChangedAsync` (in-app + SMS) via
`UpdateOrderStatusCommandHandler`.

Status: ☐ هدش یزاس‌هدای پ (in-app read/unread; SMS log). ☐ No email channel implemented (notification types
reference "Email" in some DTOs but no email sender exists).

14. SMS

Architecture: `ISmsService` selected by `Sms:Provider`:

- `"Log"` → `LogSmsService` (logs message, always succeeds) — ****default in appsettings****.
- `"Mock"` → `MockSmsService` (no-op success).
- `"Kavenegar"` → `KavenegarSmsService` (real REST call).

Current config: `Sms:Provider=Kavenegar` is set in User Secrets (overrides appsettings `Log`). `Sms:ApiKey` and
`Sms:SenderNumber` also in User Secrets. `Sms:StoreName="پاشا وون"`.

KavenegarSmsService: `POST https://api.kavenegar.com/v1/{ApiKey}/sms/send.json` with
`receptor,message[,sender]`. Validates Iranian mobile (`^09\d{9}\$`), masks phone in logs, never logs the API key,
returns `SmsSendResult{Success, ProviderMessageId, Error}`.

Triggers (via `NotificationService`): order-placed, payment-successful, order-status-changed. Custom-doll
approve/reject/accept also call `NotifyInAppAsync` (in-app) — SMS for custom-doll is not separately triggered (only
in-app).

Retry: `RetryFailed!الغ!Job` (Hangfire, every 2 min, queue `sms`) reprocesses `SmsNotification` rows with status
`Failed`/`Queued` via the same `ISmsService`.

Current state (verified 2026-08-15): Kavenegar calls return HTTP 412 (sender line not approved) or HTTP 427
(insufficient account credit) — the provider rejects sends. The sender line `2000660110` is accepted (412 gone), but
the account lacks credit (427). So SMS delivery is currently failing at the provider; the application code is correct and
the notification row is persisted as `Failed` with the error. This is an external/account configuration issue, not an app
bug.

Status: ☐ هدش یزاس‌هدای پ (provider selection, send, persist, retry). ☐ Delivery blocked by Kavenegar account (sender
approval + credit). ☐ Provider = Kavenegar (active via User Secrets); appsettings default = Log/Mock (dev).

15. Hangfire

Hangfire uses SQL Server storage; server started with `Queues = [critical,default,notifications,sms,maintenance]` and
configurable `WorkerCount`. Global retry: `Attempts` from `Hangfire:RetryAttempts` (default 3), custom
`RetryDelaysInSeconds`, `OnAttemptsExceeded=Delete`. Dashboard at `/hangfire` protected by
`AdminHangfireAuthorizationFilter` (shared `DashboardAccessKey` from User Secrets or Admin role).

Recurring jobs (registered in `ProgramHelpers.ConfigurePipeline`):

Job	Purpose	Schedule	Queue	Data	Failure behavior
release-expired-reservations	Release reservations past 15-min expiry	`*/5 * * * *` (5 min)	critical	ReleaseExpiredReservationsJob`	retry (global)
`rebuild-fts-catalog`	Rebuild SQL full-text catalog	`0 3 * * *` (daily 3am)	maintenance	`RebuildFtsCatalogJob`	retry
`retry-failed-notifications`	Resend Failed/Queued SMS	`*/2 * * * *` (2 min)	sms	`RetryFailed!الغ!Job`	retry
`inventory-health-check`	انومالی ی دوجوم check	`*/30 * * * *` (30 min)	maintenance	`HealthCheckJob`	retry
admissions-reminder	admissions-reminder (sets `ReminderSentAt`)	`0 * * * *` (hourly)	notifications	CustomDollRequestReminderJob`	retry

Job	Purpose	Schedule	Queue	Data	Failure behavior
ReconciliationJob	online payments disabled	`*/15 * * * *` (15 min)	critical	`ReconciliationJob`	retry

Status: ☐ هدش یزاس هداپ (6 recurring jobs). Background/enqueue ad-hoc jobs: `IReservationScheduler` (Hangfire) schedules expiry; events published to stub (no real consumer). No separate one-off "notification" enqueue beyond the retry job.

تیریدم لنپ 16.

Pages (`/admin/(panel)/...`): dashboard, products, categories, orders, users, inventory, reviews, notifications (SMS log), discounts, banners, custom-doll-requests. Plus `/admin/login`.

Backend admin endpoints (`RequireAuthorization("AdminOnly")`):

- `GET /api/admin/dashboard` — totalUsers, totalامشرافس, pendingامشرافس, revenue (Delivered+Shipped), dailyRevenue (7d), recentامشرافس(5).
- `GET /api/admin/orders`, `GET/PUT /api/admin/orders/{id}/status` — list + status transition (enforced).
 - `GET /api/admin/inventory` — inventory transaction ledger.
- `GET /api/admin/reviews` — moderation list (delete via `DELETE /api/reviews/{id}`).
 - `GET /api/admin/notifications/sms` — SMS log.
- `GET/POST/PUT/DELETE /api/admin/discounts` — discount CRUD.
- `GET/POST/PUT/DELETE /api/admin/banners` — banner CRUD.
- `GET/POST /api/admin/custom-doll-requests[...]/approve` + `/reject` — custom-doll workflow.
- `GET/POST/PUT/DELETE /api/users` — user CRUD.

All admin pages verified to call matching backend endpoints (see `lib/admin-api.ts`).

Status: ☐ هدش یزاس هداپ (all modules wired FE↔BE).

ریواصت تیریدم 17.

Endpoints (`ImagesEndpoints`):

- `POST /api/images/upload` (RequireAuthorization) — multipart `file, folder?, category?` → `UploadImageCommand` → `LocalImageStorage` writes to `wwwroot/images` and returns URL. Antiforgery disabled for this endpoint (client sends bearer token).
 - `DELETE /api/images/{\*\*publicId}` (AdminOnly).

Storage: `IImageStorage` → `LocalImageStorage` (local filesystem under `wwwroot/images`; `ImageStorageOptions` from config). No cloud storage (S3/Azure Blob) in running config.

Usage: product images and custom-doll request images are URLs (the custom-doll FE uses `ImageUploader` → `/api/images/upload` then stores the returned URL). Product admin also accepts image URLs (and optional upload).

Validation: `UploadImageCommandHandler` + `LocalImageStorage` — file presence checked; no explicit file-type/size limit was verified in the handler (⚠ potential gap — validation may be minimal). `publicId` in delete route maps to the stored path/key.

Status: ☐ هدش یزاس هداپ (upload/delete, local storage). ⚠ File-type/size validation not confirmed; no cloud backend.

18. یشرافس کسورع تساوخر د

Workflow (verified FE + BE):

1. Customer (authed) → `/custom-doll-request` → upload image (via `ImageUploader` → `/api/images/upload`) + description → `POST /api/custom-doll-requests` → status `PendingReview`.
2. Customer views `/custom-doll-requests` (own) + detail; can `POST /api/custom-doll-requests/{id}/accept` only when `Approved` (status → `CustomerAccepted`, notifies admin in-app).
3. Admin → `/admin/custom-doll-requests` → `POST .../approve` (sets Price + AdminMessage, status `Approved`, notifies customer in-app) or `.../reject` (status `Rejected`, notifies customer).
4. `CustomDollRequestReminderJob` (hourly) reminds admins of aged `PendingReview` requests (sets `ReminderSentAt` to avoid repeat).

Statuses: `PendingReview` → `Approved` → `CustomerAccepted` (and `Rejected`). No automatic "production/delivery" status beyond `CustomerAccepted` (△ production stage not modeled as a distinct status). Currency = Toman.

Status: □ یراس هداپ (submit/review/approve/reject/accept/remind). △ No production/delivery sub-status; SMS not sent for custom-doll (in-app only).

19. یراسه نابزدنچ

Approach: Frontend-only, hardcoded translation maps in `lib/translations.ts` (`fa`/`en`/`ar` dictionaries) + `lib/structured-translations.ts` + `lib/admin-i18n.ts`. `lib/locale-context.tsx` provides `useLocale()` (current locale, `dir` rtl/ltr). Supported locales: `fa` (rtl), `en` (ltr), `ar` (rtl). Default appears to be Persian.

Language selector: FE component switches locale (persisted client-side); `globals.css` + `dir` attribute handle RTL.

Backend: No translation/resource system. All server messages are hardcoded Persian strings in C# (`InvalidOperationException` messages, SMS templates). There is no Admin language-management module and no DB-stored translations.

Status: □ FE i18n (fa/en/ar, RTL). □ Backend localization / admin translation management NOT IMPLEMENTED. Missing-translation handling: key falls back to the key string (△ if a key is absent in a locale map).

20. لوپ فیک

Code present: `لوپ فیک` entity (Balance, Currency=IRT, Credit/Debit), `لوپ فیک Transaction` ledger, `لوپ فیک Service`, `Charge لوپ فیک Command`/`Verify لوپ فیک Charge Command`, `لوپ فیک Endpoints` (`GET /api/wallet`, `POST /api/wallet/charge`, `POST /api/wallet/verify`).

Policy gate: `لوپ فیک Policy` تخرپ. `لوپ فیک Enabled` is false (appsettings). `لوپ فیک Endpoints` return 403 Forbidden when disabled. The FE `/wallet` page exists but the API blocks it.

Status: □ CONFIGURED BUT DISABLED (`لوپ فیک Enabled=false`). Refund/charge flows exist in code but are unreachable while disabled. NOT a separate "not implemented" — the implementation exists, it is switched off by policy.

سبب اتید یاهه ن م اد 21.

Entities (NovaShopDb, SQL Server LocalDB):

Users: `User` (Id, Username[unique], Email, PasswordHash[PBKDF2], FirstName, LastName, PhoneNumber, Role[Admin|Customer], IsActive, CreatedAt) → اهش رافس دیرخ دبس، WishlistItems. `PhoneNumber` has a unique index (`20260813112819\_AddUserPhoneUniqueIndex`).

Products: `Product` (Name, Description, Price, OriginalPrice, ImageUrl, Rating, Stock, CategoryId, ReservedQuantity, ReservedUntil) → ProductImage, ProductColor, Review, OrderItem. `Category` (Name, Description, ImageUrl, ParentCategoryId?). `ProductImage` (Url, AltText, DisplayOrder, IsPrimary, ProductColorId?). `ProductColor` (Name, HexCode, Stock, IsActive, Price, Images). `Banner`.

اهش رافس: `Order` (UserId, TotalAmount, Status, اسرا لاسرا Method/Cost/Address, تخادرب Method, تخادرب Status, TrackingCode/Number, IdempotencyKey, ReservationExpiresAt, Discount*, timestamps) → OrderItem, تخادرب, OrderStatusHistory. `OrderItem` (ProductId, ProductColorId, ColorName, Quantity, UnitPrice). `تخادرب` (Amount, تخادرب Method, Status, TransactionId). `OrderStatusHistory` (From/To status, Note, ChangedByUser/Role, ChangedAt). `تدوجوم Transaction` (ProductId, OrderId, Type, Quantity, StockBefore/After, Reference).

`ل وپ فیک` (UserId, Balance, Currency), `ل وپ فیک/ستخادرب Transaction`.

(above) `تدوجوم Transaction` ی: دوجوم

اهن العا: `AppNotification` (UserId, OrderId?, CustomDollRequestId?, Type, Channel, Title, Message, Status, IsRead, ReadAt, CreatedAt). `SmsNotification` (OrderId?, PhoneNumber, EventType, Message, Provider, Status[Sent|Failed|Queued], ProviderMessageId, Error, SentAt, CreatedAt).

Custom: `CustomDollRequest` (UserId, ImageUrl, Description, Status, Price, Currency, AdminMessage, ReviewedBy, timestamps, ReminderSentAt).

Reviews: `Review` (ProductId, UserId, Rating, Comment, CreatedAt).

Discounts: `Discount` (Code, Type[Percentage|Fixed], Value, MinOrderAmount, ValidFrom, ValidTo, IsActive, UsageCount, MaxUsage), `DiscountType` enum.

Wishlist: `WishlistItem` (UserId, ProductId, ...).

Hangfire: `Hangfire.\*` schema (jobs, states, sets, hashes) in same DB.

API (major areas) یلک یامن 22.

Area	Endpoint	Method	Auth	Role
Auth	/api/auth/login	POST	Anon	any
Auth	/api/auth/register, /register/verify, /register/resend	POST	Anon	any
Auth	/api/auth/otp/request, /otp/verify	POST	Anon	any (existing user)
Auth	/api/auth/refresh	POST	Anon	any
Auth	/api/auth/logout	POST	Auth	any
Auth	/api/auth/check-mobile	POST	Anon	any
Products	/api/products, /{id}, /search, /suggestions	GET	Anon	any
Products	/api/products	POST	Auth	Admin
Products	/api/products/{id}	PUT/DELETE	Auth	Admin
Categories	/api/categories, /{id}	GET	Anon	any
Categories	/api/categories	POST	Auth	Admin
Categories	/api/categories/{id}	PUT/DELETE	Auth	Admin

Area	Endpoint	Method	Auth	Role
دیخ دس	/api/cart	GET/POST/DELETE	Auth	Customer
دیخ دس	/api/cart/items/{id}	PUT/DELETE	Auth	Customer
امش رافس	/api/orders	POST (create)	Auth	Customer
امش رافس	/api/orders	GET	Auth	Customer (own)
امش رافس	/api/orders/{id}	GET	Auth	Owner/Admin
امش رافس	/api/orders/{id}/cancel	POST	Auth	Owner
امش رافس	/api/orders/{id}/return-request	POST	Auth	Owner
امش رافس	/api/orders/{id}/pay	POST	Auth	Owner
امش رافس	/api/orders/{id}/refund	POST	Auth	Admin
ستخ ادرب	/api/payments/verify	POST	Anon	callback
ستخ ادرب	/api/mock-gateway/...	POST/GET	Anon	dev
اپی/ویلیت	/api/wallet, /wallet/charge, /wallet/verify	GET/POST	Customer	403 if Enabled=false)
اهن ال عا	/api/notifications, /unread-count	GET	Auth	Owner
اهن ال عا	/api/notifications/{id}/read, /read-all	POST	Auth	Owner
Wishlist	/api/wishlist, /wishlist/...	GET/POST/DELETE	Auth	Owner
Reviews	/api/reviews	POST	Auth	Customer
Reviews	/api/reviews/{id}	DELETE	Auth	Admin
Images	/api/images/upload	POST	Auth	any
Images	/api/images/{publicId}	DELETE	Auth	Admin
Banners	/api/banners	GET	Anon	any
Banners	/api/admin/banners	CRUD	Auth	Admin
Custom Doll	/api/custom-doll-requests	POST/GET	Auth	Owner (POST), Owner (GET own)
Custom Doll	/api/custom-doll-requests/{id}/accept	POST	Auth	Owner
Custom Doll	/api/admin/custom-doll-requests[/...]	GET/approve/reject	Auth	Admin
Users	/api/users, /{id}	CRUD	Auth	Admin
/api/admin/dashboard, /ordersAdminInventory, /reviews, /notifications/sms		GET	Auth	Admin
Admin	/api/admin/orders/{id}/status	PUT	Auth	Admin
Admin	/api/admin/discounts	CRUD	Auth	Admin
Discounts	/api/discounts (validate/apply)	query/cmd	Auth	(apply=Customer, CRUD=Admin)

23. API Gateway

Responsibility (YARP reverse proxy, `NovaShop.ApiGateway`):

- Forwards all `/api/\*\*` and `/images/\*\*` to backend (`http://localhost:5003`).
- ****تویوہ زارح****: validates JWT (`Jwt:Key` from User Secrets, shared with API). Per-route **تویوہ زارح** `AllowAnonymous` allows public routes (auth/login, register, verify, products, categories, images, etc.); `/api/auth/logout` and most `/api/admin/\*\*` require auth at the gateway (but the API also re-checks `AdminOnly`).
- ****Rate limiting****: global fixed-window 600 req/min/IP (`RateLimiting:Global`) + 429 on exceed. (The API has its ***own*** finer-grained **RateLimitingMiddleware**: auth=5/min, admin=500/min, authed=300/min, default=100/min, window 60s — applied at the API, after the gateway.)
 - ****CORS****: `AllowOrigins=[http://localhost:3000]`, AllowAnyMethod/Header, AllowCredentials.
 - ****Correlation-Id**** middleware (logs method/path/client IP).
 - ****Health****: `/health`.

What belongs to Gateway vs Backend:

- Gateway: edge auth (JWT validate), global rate limit, CORS, routing.
- Backend: business logic, per-route authorization (`AdminOnly`), per-endpoint rate limit, antiforgery, Hangfire, DB.

Backend functionality that should NOT be in Gateway: order/cart/product logic (correctly absent — gateway is pure proxy). The gateway does not transform request bodies or implement business rules.

⚠ Note: gateway rate limit (600/min) is much looser than the API's (100/min default); the effective limit is the stricter API one. Both apply.

24. Frontend تاحفص

Public: `/` (home, hero, product grid, banners), `/products` (list/search/filter), `/products/[id]` (detail + add-to-cart + reviews), `/login`, `/register`, `/admin/login`.

Customer (auth required): `/cart`, `/checkout`, `/orders`, `/orders/[id]`, `/notifications`, `/profile`, `/wishlist`, `/wallet` (disabled 403), `/custom-doll-request`, `/custom-doll-requests`, `/custom-doll-requests/[id]`.

Admin (`/admin/(panel)/...`): dashboard, products, categories, orders, users, inventory, reviews, notifications, discounts, banners, custom-doll-requests.

Navigation: customer layout (header with cart badge, lang switch, account/logout) ↔ admin layout (sidebar).
Login/register redirect to `/products`. باسح هیوست redirects to `/orders/[id]` on success.

APIs used: all via `API\_GATEWAY\_URL` (= `http://localhost:5100`). Public reads use SSR `fetch` (revalidate cache);
authed writes use `authFetch` with bearer cookie.

25. Workflow راکوبسک (verified)

Registration: `/register` (phone+user+pass) → OTP SMS → `/register/verify` → user created (Customer) + JWT →
`/products`.

Login/OTP: password or OTP → JWT cookie. Admin uses same login, role-gated nav.

Browse Product: home/products → `GET /api/products` (filter/search/sort) → detail `GET /api/products/{id}` (images, colors, reviews).

Add To دبس: detail → `POST /api/cart` (requires color if product has colors) → DB cart.

باسح هیوست: `/cart` → `/checkout` (address, shipping method, discount) → `POST /api/orders` (InPerson) → stock reserved, cart deleted → `/orders/[id]`.

Order Creation: handler reserves stock, creates Order/تخادرپ/History/ی دوجوم Transaction, notifies (in-app + SMS attempt). Idempotency via header.

تخادرپ: online disabled; InPerson only. `POST /api/orders/{id}/pay` exists but not used in normal flow.

لاسرا: customer picks POST/COURIER/PICKUP (cost from FE). Admin marks Shipped (assigns tracking) / Delivered.

Delivery: admin `PUT /api/admin/orders/{id}/status` → Delivered; SMS sent.

Order Cancellation: customer `POST /api/orders/{id}/cancel` → Cancelled; stock released/restored; SMS.

تالوصحم تیردم: admin CRUD via `/admin/products` → `POST/PUT/DELETE /api/products` (with images/colors).

Image Upload: `ImageUploader` → `POST /api/images/upload` → local `wwwroot/images` URL.

Custom Doll Request: submit (image+desc) → PendingReview → admin approve (price)/reject → customer accept → CustomerAccepted; reminders hourly.

Notification: in-app created on order/status/custom-doll events; SMS attempted via Kavenegar (currently failing at provider).

SMS: order-placed/payment/status → `NotificationService` → `KavenegarSmsService` → provider; failed rows retried every 2 min by Hangfire.

Admin Management: dashboard stats, order status changes, user CRUD, inventory ledger, review moderation, discount/banner CRUD, SMS log, custom-doll review.

توانا کما تی عضو لودج 26.

	Functionality	Frontend	Backend	DB	Admin	Status
	Register (OTP)	☐	☐	☐	—	☐
	Login (password)	☐	☐	☐	—	☐
	Login (OTP)	☐	☐	☐	—	☐
	Logout	☐	△ (no server revoke)	—	—	△
	Browse/Search/Filter products	☐	☐	☐	—	☐
	Product detail + images + colors	☐	☐	☐	—	☐
	Add/Update/Remove/Clear cart	☐	☐	☐	—	☐
	باسح هیوست (InPerson)	☐	☐	☐	—	☐
	Online payment	☐ (UI tab)	☐ (validator blocks)	☐	—	☐ DISABLED
	Create order	☐	☐	☐	—	☐
	Order list/detail (customer)	☐	☐	☐	—	☐
	Cancel/Return order	☐	☐	☐	—	☐
	Order status lifecycle	☐	☐ (validated)	☐	☐	☐
	لا سرا POST/COURIER/PICKUP	☐	☐	☐	—	☐
	لا سرا cost engine	△ (FE only)	☐	—	☐	△
	ردوم reserve/release/ledger	—	☐	☐	☐	☐
	In-app notifications	☐	☐	☐	☐ (log)	☐
	SMS (Kavenegar)	—	☐	☐	☐	△ (provider 412/427)
	SMS retry (Hangfire)	—	☐	☐	—	☐
	Custom doll request	☐	☐	☐	☐	☐
	Reviews	☐	☐	☐	☐	☐
	Wishlist	☐	☐	☐	—	☐
	لوپ فیکی	☐ (page)	☐ (code)	☐	—	☐ DISABLED (policy)
	Discounts	☐	☐	☐	☐	☐
	Banners	☐	☐	☐	☐	☐
	Categories CRUD	☐	☐	☐	☐	☐
	User CRUD	☐	☐	☐	☐	☐
	Admin dashboard	☐	☐	☐	☐	☐
	Image upload (local)	☐	☐	☐	☐ (delete)	☐
	لا سرا هاناب ردنچ (fa/en/ar, RTL)	☐	☐	☐	☐	△ FE only
	Admin language mgmt	☐	☐	☐	☐	☐
	Email notifications	☐	☐	☐	☐	☐
	MassTransit/RabbitMQ	—	☐ (stub)	—	—	☐ DISABLED
	Real PSP integration	☐	☐ (mock)	—	—	☐
	OpenTelemetry export	—	☐ (OTLP)	—	—	△ (no collector)
	Hangfire dashboard	—	☐ (protected)	—	☐	☐
	Rate limiting (gateway+api)	—	☐	—	—	☐

Functionality	Frontend	Backend	DB	Admin	Status
JWT + role authz	☐	☐	☐	—	☐

27. دلغف یاهت ی دودحم

1. **Online payment disabled** (Policy.OnlineEnabled=false) — checkout is InPerson only; POST /api/orders/{id}/pay / /payments/verify unreachable in normal flow.
2. **disabled** (Enabled=false) — /api/wallet returns 403; refund-to-wallet path unreachable.
3. **SMS delivery failing** — Kavenegar returns 412 (sender not approved) / 427 (insufficient credit). App code correct; provider/account issue. (Status as of 2026-08-15: sender `2000660110` accepted, account lacks credit → 427.)
4. **cost in FE, not backend** — rates hardcoded in checkout/page.tsx; backend accepts client-sent shippingCost without recomputation; admin cannot configure rates.
5. **MassTransit/RabbitMQ disabled** — events go to a no-op stub; OrderCreatedConsumer / ProductCreatedConsumer / StockReservedConsumer not wired.
6. **OTP/PendingRegistration in-memory** — lost on API restart; not multi-instance safe.
7. **Logout is client-only** — no server-side token revocation/refresh-token store.
8. **No email channel** — notification DTOs hint at Email but no sender implemented.
9. **is FE-only** — backend messages hardcoded Persian; no admin translation UI; missing keys fall back to key string.
10. **Image validation** — file type/size limits not verified in UploadImageCommandHandler / LocalImageStorage.
11. **ConfirmReservation** not called on Paid — reserved stock not zeroed on payment; inventory math relies on reservation bookkeeping that isn't fully closed (ledger still records correctly).
12. **OpenTelemetry** configured but no OTLP collector in this environment (exports nowhere).
13. **No product "status" field** — availability derived from Stock>0; no explicit Active/Inactive/Draft toggle beyond stock.
14. **Custom-doll production stage** — no distinct "in production / shipped as custom" status beyond CustomerAccepted.

28. یتینما یاهت ی لباق

هدش یزاس هدا ی ☐

- **JWT** (HS256, 8h, Issuer/Audience=NovaShop, Jti). Gateway + API both validate Jwt:Key (User Secrets; throws at startup if missing).
- **Authorization**: AdminOnly policy (RequireRole("Admin")); customers blocked from admin endpoints (403); order endpoints enforce ownership.
- **Rate limiting**: gateway global 600/min/IP; API per-route (auth 5/min, admin 500, authed 300, default 100, window 60s) → 429 with Retry-After + X-RateLimit-* headers.
 - **Secret management**: Jwt:Key, Kavenegar ApiKey, SenderNumber, Hangfire DashboardAccessKey, RabbitMQ creds in .NET User Secrets (not in appsettings/appsettings.Development.json, which were scrubbed of secrets). dotnet user-secrets per project.
- **Hangfire dashboard protection**: AdminHangfireAuthorizationFilter (shared DashboardAccessKey or Admin role).
- **Input validation**: FluentValidation on all commands (product, order, auth, cart, custom-doll, discount).

- **Exception handling**: `ExceptionHandlerMiddleware` (uniform error responses; maps `UnauthorizedAccessException`→401, `InvalidOperationException`→400, etc.).
- **Anti-enumeration**: `/auth/check-mobile` returns identical shape; OTP/register errors generic.
- **SMS secret hygiene**: `KavenegarSmsService` masks phone, never logs API key; Serilog `System.Net.Http: Warning` suppresses request-URI (key) leakage.
- **Password hashing**: PBKDF2 (`Pbkdf2PasswordHasher`, 100k iter, SHA256).
- **Antiforgery**: enabled; image-upload endpoint explicitly disables it (uses bearer token instead).
- **CORS**: restricted to `http://localhost:3000`.

△ Notes/gaps:

- Logout does not revoke token (stateless JWT, no blacklist).
- Refresh-token store is in-memory (not persisted).
- Gateway `Jwt:Key` must match API's (both from User Secrets) — verified working cross-project.
- `appsettings.Development.json` `Logging` override initially shadowed the Serilog fix; resolved by adding `Serilog:MinimumLevel.Override` (this analysis found the config is now correct).

سیاهن یبایزرا 29.

What the system currently does

A working Persian/RTL handmade-doll storefront: customers register (phone+OTP) or login (password/OTP), browse/search/filter products with images and color variants, manage a DB-backed cart, check out with InPerson payment + chosen shipping method (POST/COURIER/PICKUP), and place orders that reserve inventory and trigger in-app + SMS notifications. Admins manage products, categories, orders (status lifecycle), users, inventory ledger, reviews, discounts, banners, and custom-doll requests via a full admin panel. Custom-doll requests flow submit→admin approve/reject→customer accept.

What the main user (customer) can do

Register, login, browse, search, add to cart (with color), checkout (InPerson), view/cancel/return orders, read notifications, manage wishlist, write reviews, submit custom-doll requests. ل و پ فیک is present in UI but disabled.

What the admin can do

Full CRUD on products/categories/users/discounts/banners; order status transitions (enforced state machine); inventory ledger view; review moderation; SMS log; custom-doll review (approve with price / reject); dashboard stats.

Major business workflows

Register → Browse → دیخ دبس → باسح هیوسن (InPerson) → Order (stock reserved) → Admin ships/delivers (SMS) → Cancel/return (stock restored). Custom-doll: submit → review → approve → accept.

Currently disabled

- Online payment (policy).
- ل و پ فیک (policy).
- MassTransit/RabbitMQ (default off; stub).
- Real PSP (mock gateway only).

Incomplete / gaps

- SMS delivery blocked by Kavenegar account (sender approval + credit) — app code correct.
- لاسرا cost logic lives in FE, not backend.

- OTP/pending-registration in-memory (not durable/multi-instance).
- No email channel; localization FE-only; logout no server revoke; `ConfirmReservation` not closed on Paid; OpenTelemetry has no collector.

Most important missing functionality

1. **Working SMS delivery** — resolve Kavenegar sender approval + account credit (provider/account task, not code).
2. **Online payment enablement** — wire a real PSP and flip `Policy.OnlineEnabled=true` (validator already supports it).
3. **Backend shipping-cost engine + admin-configurable rates** (move from FE hardcoded).
4. **Durable OTP/session store** (Redis) for multi-instance safety.
5. **Backend localization / admin translation management** if multi-language admin or server messages are required.

End of documentation. No source files were modified.